

DISASTER TWEETS 2026: AN AUTONOMOUS LLM-DRIVEN RESEARCH AGENT FOR BINARY TEXT CLASSIFICATION

*Built and Benchmarked on the Kaggle Disaster Tweets Challenge
under a 1-Hour CPU-Only Compute Budget*

Advanced Predictive Analytics 2025/2026

Course Project, Track A

Spring 2026

Research performed by: Niccolò Germani | Simon Gühring | Rahul Rajesh | Dominik Plöchl

Video walkthrough: youtu.be/yUgWHVomt4 | Kaggle leaderboard: [Team Tweets Master](#)

Project Goal

This project addresses Track A of the Advanced Predictive Analytics course competition, which is based on the Kaggle challenge *NLP with Disaster Tweets*. The classification objective is binary: given the text of a tweet, optionally accompanied by a keyword and location field, a model must predict whether the post refers to a real-world disaster (`target = 1`) or not (`target = 0`). The training set consists of 7,613 labelled tweets, and model performance is evaluated using the F1 score on a held-out test set of 3,263 tweets that remains inaccessible throughout development, making local validation design a critical methodological concern. Rather than addressing this task through manual model development, this project pursues a fully autonomous approach: an LLM-driven agent that proposes, generates, executes, and evaluates machine learning experiments without human intervention after launch.

Early strategy and first goals. At the beginning of the project, our first goal was not to hardcode a manual baseline model. Instead, we wanted to get the local LLM-based workflow running as early as possible. The initial development effort therefore focused on connecting a local LLM, `gemma4:e4b`, to the project pipeline and verifying it could participate in the experimentation process. The remainder of this section documents the four iterations of the agent architecture, each addressing the structural weaknesses exposed by its predecessor.

Agent Evolution

V1: Single-architecture LLM baseline. The first version was a minimal proof of concept. A locally hosted LLM (`gemma4:e4b`) was given *one large, monolithic prompt* asking it to propose, generate, train, and evaluate an experiment end-to-end in a single call. This was the main failure mode of V1: the LLM had to juggle every responsibility in one prompt, with no specialisation and no recovery mechanism. The result was unpredictable: data-loading errors, scope bugs in the generated code, no repair, no pre-flight, specification drift, indexing errors in the submission step, and a hardcoded family loop. V1 confirmed the LLM loop ran end-to-end but combined LLM unpredictability with none of LLM’s flexibility. The lesson carried into V2 was to split the work into many smaller, specialised prompts rather than one big one.

V2 Single-file autonomous multi-architecture agent. V2 gave the LLM ownership of the entire training script (the `FULL_SYSTEM` prompt) and surrounded it with safety nets: pre-flight validation and autofix before execution, a four-attempt repair loop on runtime failure, a multi-architecture sweep over a `REQUIRED_ARCHITECTURES` list, and a 60-second dry run before each full run with a 1000-second timeout. A plateau detector tracked F1 movement across a sliding window. The first working V2 ran a plain BoW pipeline (TF-IDF + logistic regression) as the baseline. A second variant, `BoW_advanced`, combined multiple TF-IDF vectorisers (word and character n-grams, separate channels for the keyword and location fields) and stacked several linear classifiers (logistic regression, calibrated linear SVM, multinomial naïve Bayes) into a single feature space. Despite the gain, V2 was still not learning: the plateau detector never fired despite five identical BoW and LSTM runs, “variation” across runs was effectively one-axis, degenerate models with accuracy = 0.0 were marked successful, and analyst feedback never closed the loop into the next spec.

V3 Multi-family modular LLM agent. V3 separated proposal from execution with a two-LLM pipeline. A *Proposer* reads the trial history and produces the next JSON spec, and a *Coder* translates it into a full training script. Each of the eight model families received its own Jinja2 template. The one-hour budget was explicitly partitioned into `SWEEP` → `OPTIMIZE` → `FINAL`: a 2,000-row sweep across all families, a focused tune of the top two using a `tune_frozen_code()` helper, and a retrain on the full 7,613 labelled rows for the submission. A surgical JSON-patch repair loop (up to 8 attempts) replaced full rewrites, and a per-family validator clamped numeric proposals to safe ranges. Best local F1: 0.8379 (DistilBERT); best Kaggle F1: 0.842 on RoBERTa across 354 logged experiments (75.4 % success rate). Three problems carried over: the order and frequency in which families were tried was still hardcoded,

degenerate successes were still mislabelled, and the final-submission script crashed with a `SyntaxError` on one launch.

V4 LLM-driven sweep planner with cross-launch memory. V4 is the current production architecture and is described in detail in §1. In one line: it replaced the hardcoded family loop with an autonomous Sweep Planner that reads a live state table under a hard Phase A / Phase B wall-clock gate, added a 20-trial cross-launch memory and a signature-hash veto to eliminate duplicate experiments across runs, specialised the LLM into five distinct roles, and pushed the final submission into a hardcoded Python tail so no LLM is invoked at submission time.

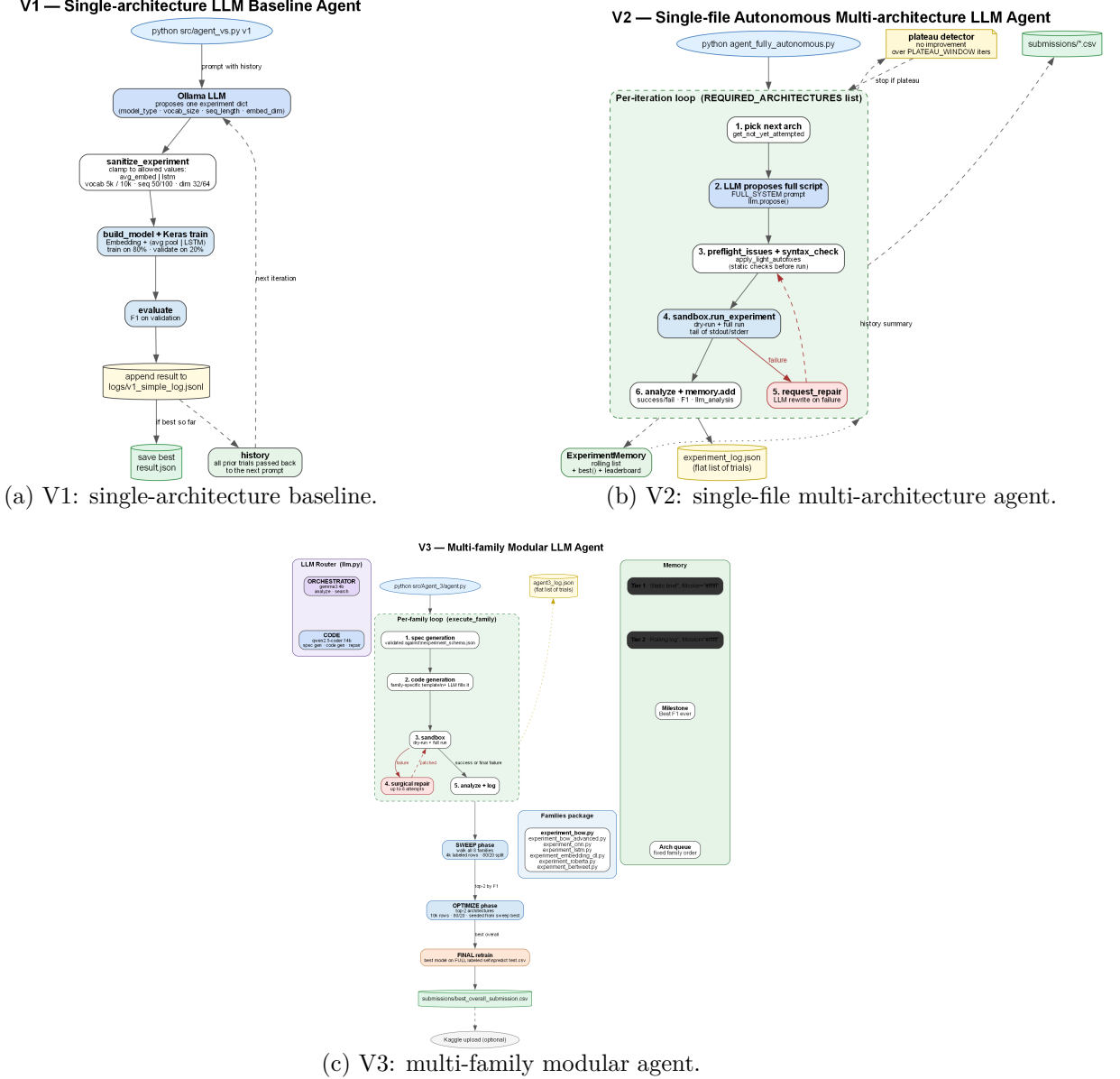


Figure 1: Architecture progression V1 → V3. Each iteration removed one category of trust placed in the LLM and replaced it with an explicit contract, a deterministic fallback, or a separated role. V4 is shown in detail in Figure 2.

1 Agent Architecture: How Our Agent Makes Autonomous Decisions

Figure 2 shows the V4 architecture. At the very top sit the two phase tags: a wall-clock controller flips the agent’s goal at 55 % of the 45-minute sweep budget. Phase A covers untried families and ignores F1, Phase B drills the current leader. The next band lists the five LLM roles; every yellow box is one call to the same locally-hosted `qwen2.5-coder:14b` model and the temperature used in that call is printed inside the box. Beneath that are the seven deterministic orchestrator steps that sit around those LLM

calls; steps 3 and 4 (purple) are the only points where the orchestrator overrides the LLM, the rest (blue) just prepare or post-process its output. The next row down is the persistence layer on the left (per-trial artefacts and a 20-trial cross-launch memory) and the final-submission pipeline on the right (the highest-F1 trial is retrained on 5,000 rows and the resulting CSV is uploaded to Kaggle). The bottom row lists the seven model families the planner can pick from.

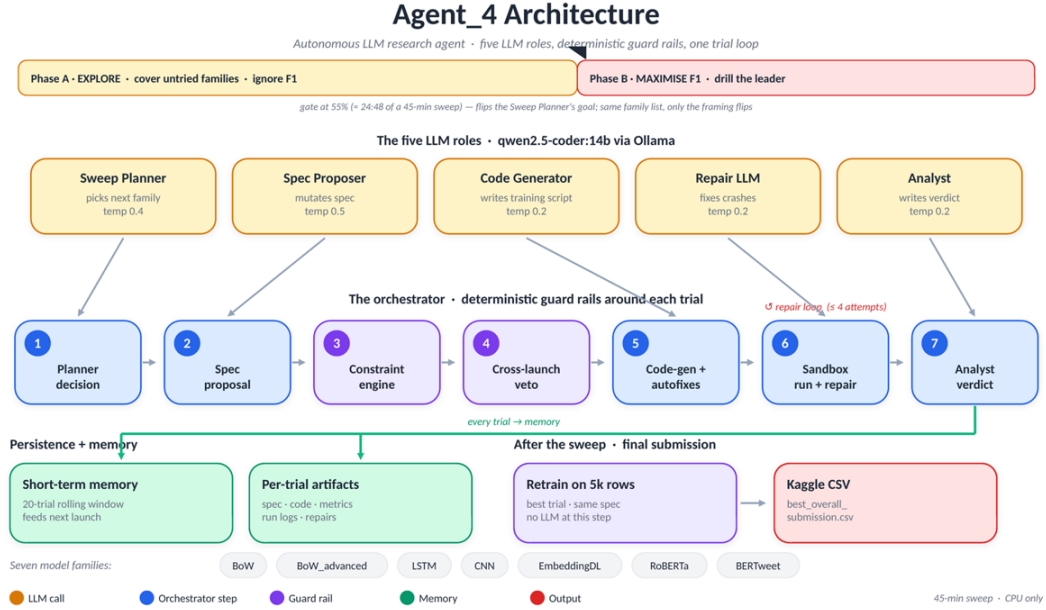


Figure 2: V4 architecture: five LLM roles, seven orchestrator steps, and the Phase A / Phase B wall-clock gate.

1.1 The five LLM roles

The same model is called in five distinct roles, each with its own system prompt and temperature. The temperature split is deliberate: creativity where it helps, determinism where it hurts.

The Sweep Planner ($T=0.4$). After every finished trial it picks the next family from the per-family state table (Table 1). The system prompt requires the *reason* field to cite one concrete number from the table; vague justifications are rejected. The planner has no way to “stop” the sweep; only the 45-minute deadline ends it.

Planner reply (18 May 2026 21:17, Phase B)

```
{
  "action": "try_family",
  "family_key": "bertweet",
  "reason": "All families have been tried this launch, so we must explore an untried family. However, since all are tried, we can re-try any of them to ensure coverage. BERTweet has the highest bestF1 (0.7730) and was successful in its attempt."
}
```

The Spec Proposer ($T=0.5$). Once the planner has chosen a family, the Spec Proposer chooses the hyperparameters. Its prompt contains the family-specific search instructions, a four-column “Prior Trials” table filtered to the chosen family (this-launch trials first, then the cross-launch memory’s last 20 entries), an *anchor* spec (the current best successful trial for that family), and the list of hyperparameters it is allowed to vary. The required JSON output has three parts: a *why* field that must reference a prior F1 and the keys being changed, an array of the changed hyperparameters, and the full proposed spec.

Spec Proposer reply for the BERTweet trial above

```
{
  "why": "Prior F1=0.7730 with max_len=104, train_batch_size=13, eval_batch_size=16, learning_rate=1.3e-5, weight_decay=0.01, num_epochs=3; changing max_len=104->102, learning_rate=1.3e-5"
}
```

```

->1.2e-5 to push above 0.7730.",
"changed_keys":["max_len","learning_rate"],
"architecture":"BERTweet",
"model_name":"vinai/bertweet-base",
"max_len":102,          "train_batch_size":13,
"eval_batch_size":16,   "learning_rate":1.2e-5,
"weight_decay":0.01,    "num_epochs":3,
"val_size":0.2,         "threshold_min":0.3,
"threshold_max":0.7,    ... }

```

The Code Generator ($T=0.2$). Its prompt is rendered from a family-specific template that injects the validated spec, the dataset context, the family’s history and a summary of earlier trials in the launch. The script it returns must sweep a 41-point threshold grid in $[0.3, 0.7]$ and print a final METRICS line that the orchestrator parses to recover the F1.

The Repair LLM ($T=0.2$). Invoked only when the training script fails. Its prompt contains the broken code, the error traceback, the last 30 lines of standard output, and a family-specific repair hint. The required output is a JSON object with a short diagnosis and one to three small textual edits (each one a replace, an insert-before, or an insert-after on an exact snippet). Up to four repair attempts per trial.

The Analyst ($T=0.2$). Runs at the end of every trial. It receives the spec, the hypothesis, the metrics and the tails of stdout and stderr, and writes a structured three-to-five-sentence verdict.

Analyst verdict closing the same trial (verbatim)

CONCLUSION: The hypothesis was refuted. The experiment aimed to improve the F1 score above 0.7730 by adjusting the `max_len` and `learning_rate`. However, the new configuration resulted in an F1 score of 0.7726, which is slightly below the original baseline.

WHAT WORKED: The training runtime was efficient, with a `train_samples_per_second` of 21.22, indicating that the model trained relatively quickly without significant issues.

WHAT FAILED: The primary failure mode was the inability to achieve the desired improvement in F1 score. Despite minor adjustments, the performance did not meet the hypothesis’s target.

NEXT MOVE: For the next trial, consider further exploring other hyperparameters such as `weight_decay` or experimenting with different architectures that might better suit the dataset characteristics.

The verdict is appended to the cross-launch memory and to the trial’s log, so the next Planner call’s state table reflects it and the next Spec Proposer call’s prior-trials table embeds it directly.

1.2 The seven orchestrator steps

Each LLM call is bracketed by a deterministic orchestrator step the LLM cannot bypass. **Step 1, Planner decision**, runs the LLM’s family pick through an eligibility filter that hard-skips families which have crashed twice in a row without ever succeeding, families stuck in a tight F1 band, families that have hit the cap of five attempts, and families whose estimated cost exceeds the remaining budget. If the LLM picked one of those, the cheapest eligible family is substituted with a logged fallback. **Step 2, Spec proposal**, parses the JSON reply and clamps every numeric hyperparameter to the safe range that family allows.

Steps 3 and 4, the Constraint engine and Cross-launch veto, are the safety nets that keep the trial honest *only when the LLM fails to declare its changes correctly*. The constraint engine reads the array of hyperparameters the LLM declared it was changing and produces a spec where only those keys take new values, with every other key reset to the anchor. It also enforces a minimum of two hyperparameter changes per trial; if the LLM proposed fewer, it extends the move and tags the recorded hypothesis as “orchestrator-added”. The cross-launch veto computes a signature over the tunable hyperparameters and refuses any spec whose signature exactly matches a prior-launch trial of the same family. In practice the LLM proposes a clean, well-declared spec on most trials; these guards fire on the minority of cases.

Step 5, Code-gen + autofixes, calls the Code Generator and then runs a family-specific autofix pass that mechanically repairs a small set of known recurring mistakes. A static pre-flight check then verifies the required elements are present (the right classifier, the threshold sweep, the metrics print) and rejects known banned patterns.

Step 6, Sandbox, runs the script in a CPU subprocess in two stages. A 60-second dry run on a tiny head of the data fails fast on syntax errors and missing imports. The full run trains on 1,600 rows and validates on 400 rows (an 80 / 20 split of the fixed 2,000-row sweep sample, with a fixed random seed) under a 1000-second timeout. The same sample is reused across every trial in the launch, which is what makes F1 numbers comparable.

Step 7, Analyst, parses the metrics line and classifies the trial outcome into one of six buckets: success, degenerate success, code-generation failure, timeout, training crash, or no-metrics (script ran but never printed the expected line). A typical state table the planner sees after a handful of trials looks like Table 1 in §1.4: one row per family with columns for attempts, successes, best F1, last F1, outcome, wall time, cost estimate and an eligibility flag.

1.3 Persistence, families, and final submission

Every trial produces a folder of audit artefacts (rendered prompts, validated spec, training script, repair attempts, metrics, the run log with the Analyst’s verdict). Across launches the agent keeps a *short-term memory* file: a rolling window of the 20 most recent trials with their family, hypothesis, spec, outcome, F1, accuracy, best threshold and analyst verdict. This file is read at launch start by the Spec Proposer’s per-family filter, so every revisit sees the prior-launch evidence for that family; the Planner deliberately does *not* see it. The seven families cover every approach in the project handout’s NLP table (full descriptions in §4). When the 45-minute deadline is reached, the highest-F1 trial wins; the agent then retrains its script on a 5,000-row sample (capped by the remaining wall-clock budget rather than the full set), predicts the full 3,263-row test set, and writes the Kaggle submission CSV.

1.4 What makes the agent’s decisions genuinely autonomous

The agent’s behaviour is determined at three points: which family to try, which hyperparameters to try inside it, and whether to abandon a broken script or try to fix it. Each is LLM-driven where reasoning helps and orchestrator-enforced where the LLM cannot be trusted to be consistent. Three mechanisms are unique to V4 and are what make the loop genuinely autonomous rather than a wrapper around random search.

Decisions grounded in evidence, not in priors. Both the Sweep Planner and the Spec Proposer must ground their choices in evidence from the current launch (the Spec Proposer also sees the last 20 trials of the chosen family from the cross-launch memory, filtered per-family). The planner reads the per-family state table (Table 1) and picks one row from the eligible column; every row carries five hard facts: attempts, successes, best F1, last F1, and outcome of the most recent run. The Spec Proposer is constrained the same way: it sees a four-column prior-trials table, is given the *anchor* (the current best successful trial for that family, not the family default), and must mutate it. Three hard constraints force a real experiment: at least two hyperparameter changes versus the anchor, an explicit declaration of which keys it is changing, and a justification field that cites a prior F1.

family	att	ok	bestF1	lastF1	outcome	wall	cost	eligible
bertweet	2	2	0.7730	0.7726	success	785s	500s	no
bow	2	1	0.7043	n/a	code_gen_failed	215s	100s	yes
bow_advanced	2	2	0.7247	0.7247	success (stagnant)	215s	200s	no
cnn	2	1	0.6582	n/a	code_gen_failed	341s	200s	no
embedding_dl	1	1	0.6932	0.6932	success	148s	240s	no
lstm	1	1	0.5965	0.5965	success	129s	200s	no
roberta	2	1	0.7859	n/a	code_gen_failed	526s	500s	no

Table 1: Real per-family state table from a Phase B call (18 May 2026, 21:34). The `eligible` column is computed by the orchestrator from cost, attempts and plateau status; the LLM picks only from rows where `eligible=yes`.

The constraint engine keeps the hypothesis honest. When the LLM declares it is changing two specific hyperparameters (say, learning rate and sequence length), the constraint engine takes the anchor

spec, overwrites only those two values with the LLM’s new ones, and resets every other hyperparameter to the anchor. Before the engine existed, the analyst’s verdicts were being computed against a different experiment than the one the LLM described in its hypothesis; the engine eliminates that gap. It also enforces the floor of two changed hyperparameters: if the LLM proposed fewer, the engine extends the move and tags the recorded hypothesis as “orchestrator-added”. In practice the LLM proposes a clean, well-declared spec on most trials; the constraint engine fires on the minority of cases where the LLM tries to silently change more keys than it claimed.

Surgical repair instead of full regeneration. When the training script crashes, the agent does *not* regenerate it from scratch. The Repair LLM returns a JSON object with one to three small textual edits which the orchestrator applies mechanically; if any target snippet is not found in the source the patch is rejected and another repair attempt is requested, up to four. Almost every successful repair we observed was a one-line edit, exactly the kind of fix a human reviewer would make.

The combination closes the feedback loop: every trial’s verdict becomes evidence the next trial reasons against: a new row in the planner’s state table, a new row in the spec proposer’s prior-trials table, a new signature in the cross-launch veto. The agent’s “smartness” is in this loop, not in any single decision.

2 Experiment Log Analysis

2.1 Reliability and per-family score distribution

Across the 557 logged trials across V1 to V4, 437 produced a usable F1. Success here means the training script ran end to end *and* printed a parseable metrics line with $F1 \geq 0.4$ (the degenerate one-class predictor case is treated as failure); the resulting success rate is 78 %. Figure 3 shows the failure rate per family and Figure 4 pairs the mean local F1 of each family with its single best run.

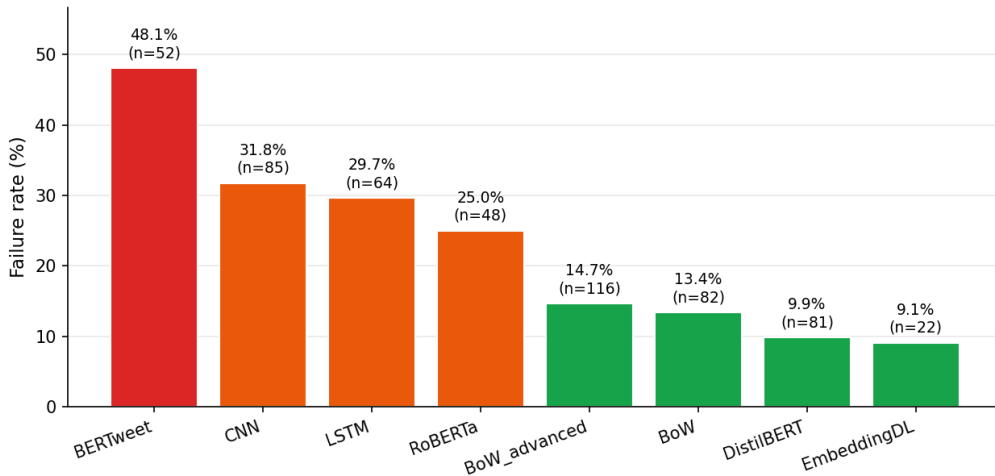


Figure 3: Failure rate per family across all 557 trials (V1 to V4 pooled). BERTweet’s higher rate reflects the code-generation LLM’s recurring difficulty with its slow tokenizer and tweet-normalisation requirements; the BoW variants, RoBERTa and DistilBERT all sit below 25 %.

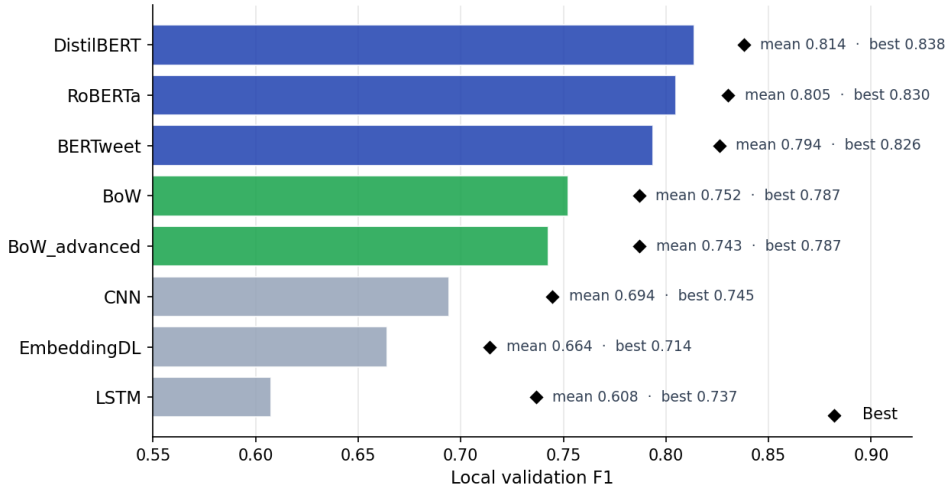


Figure 4: Mean local validation F1 per family (bars), with the family’s best single run marked as a black diamond. The three transformer families cluster between 0.79 and 0.82 mean; BoW variants around 0.75; the from-scratch deep models (CNN, EmbeddingDL, LSTM) below 0.72.

2.2 Score progression over iterations and what worked

Figure 5 plots every successful F1 across the agent’s 437 successful runs in chronological order, with transformer-family runs in red. The red cluster sits almost entirely above 0.80; the grey cloud rarely crosses it. The visible drift towards more red dots in later iterations is the agent leaning increasingly on transformer families as launches accumulate.

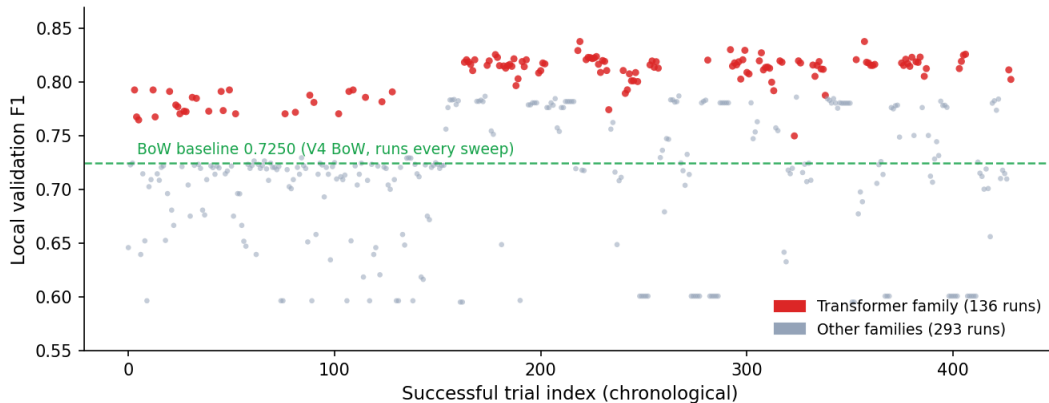


Figure 5: Every successful local F1 the agent recorded, in chronological order. Red dots are transformer-family runs (DistilBERT, RoBERTa, BERTweet); grey dots are every other family. Dashed green line is the V4 BoW baseline at 0.7250.

Two things worked clearly: pretrained transformer fine-tuning (every transformer family hit 0.82 or higher on its first successful run) and threshold tuning on a 41-point grid in $[0.3, 0.7]$ (added 0.5 to 1.5 F1 points over the default 0.5 cutoff for every family). Training deep architectures from scratch (LSTM, CNN, EmbeddingDL) plateaued in the 0.61 to 0.75 band; 7,613 labelled tweets is far below what is needed to compete with a pretrained backbone. DistilBERT held the all-time best local F1 of 0.8379 but its per-run cost on CPU was too high to keep inside the V4 per-family budget once dedicated RoBERTa and BERTweet hooks existed.

3 Comparison with the BoW Baseline

3.1 The BoW baseline and per-version progression

The baseline is the BoW family itself: a TF-IDF vectoriser with word n-grams feeding a logistic regression. It is not a separate hand-coded script; it is one of the seven families the planner picks from. The

honest reference for the agent’s current performance is the V4 BoW best of 0.7250 across 26 successful V4 BoW runs (its best across the full V1 to V4 pool was 0.7869, but V3 had access to larger training samples; under V4’s fixed 2,000-row sweep budget the same family caps out at 0.7250). This V4-relative number is the floor we plot in every chart: it is what BoW actually delivers inside the same compute budget every other V4 family is judged on. Figure 6 shows headline F1 per version against this baseline, and Table 2 gives the trial counts behind those bars.

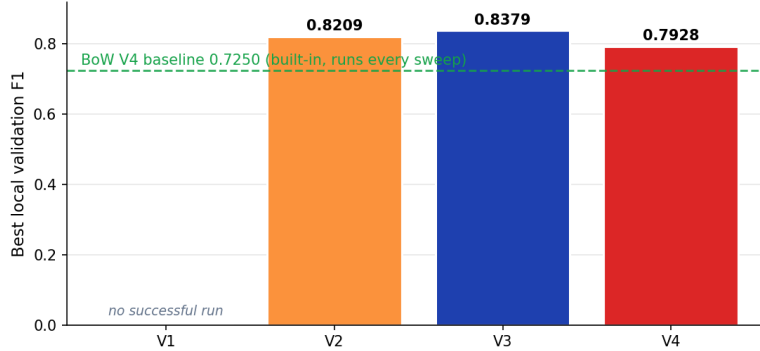


Figure 6: Best local F1 per version vs. the V4 BoW baseline at 0.7250. V1 produced no successful run; V4 drops below V3 because of the smaller per-trial sample (§3.2).

Version	Trials	Successful	Failure rate	Best local F1	Best family
V1	3	0	100%	n/a	n/a
V2	35	34	2.9%	0.8209	DistilBERT (Transformer)
V3	296	246	16.9%	0.8379	DistilBERT (Transformer)
V4	223	157	29.6%	0.7928	RoBERTa

Table 2: Per-version trial counts and best local validation F1.

3.2 Per-family ranking across all versions, and why V4 looks worse locally

Restricting the per-family ranking to *V4 successful runs only* (Table 3) makes the comparison apples-to-apples: every number in the table was produced under the same 45-minute sweep budget, with the same training data (1,600 rows train and 400 validation, taken from a fixed 2,000-row sample at the start of the launch). The two transformer families lead at ≈ 0.79 , the BoW variants sit at 0.72 to 0.73, and the from-scratch deep families (CNN, EmbeddingDL, LSTM) underperform the BoW baseline despite using deep learning. Comparing V4 to V3 head-to-head, the drop is systematic across every family, not a regression in one of them: BERTweet $0.8261 \rightarrow 0.7914$, RoBERTa $0.8303 \rightarrow 0.7928$, BoW $0.7869 \rightarrow 0.7250$, CNN $0.7446 \rightarrow 0.6753$, LSTM $0.7366 \rightarrow 0.6207$. V3 had access to longer per-trial training and to the now-deprecated DistilBERT family that produced the V3 winner. The trade-off is deliberate: V3 could spend an entire launch drilling DistilBERT, while V4 budgets across all seven families plus a dedicated final-retrain step.

Family	Best F1	Mean F1	V4 successful runs
RoBERTa	0.7928	0.7832	12
BERTweet	0.7914	0.7781	16
BoW_advanced	0.7296	0.7214	59
BoW (<i>baseline</i>)	0.7250	0.7105	26
EmbeddingDL	0.7027	0.6588	18
CNN	0.6753	0.6556	14
LSTM	0.6207	0.5992	9

Table 3: Best and mean local validation F1 per family, restricted to V4 successful runs only ($F1 \geq 0.3$ filter). DistilBERT is excluded because it was deprecated before V4.

3.3 Kaggle submission history

Figure 7 plots our 22 Kaggle submissions in chronological order, colour-coded by agent version. The all-time best is 0.84216 from submission 9, a manual 16-epoch BERTweet retraining built on top of an agent-discovered BERTweet spec on GPU (the architecture choice was the agent’s; the 3-epoch cap was lifted manually). The best fully-agent-driven submission is 0.84002 (submission 7, V3-era BERTweet); V4’s best is 0.83052 (submission 20).

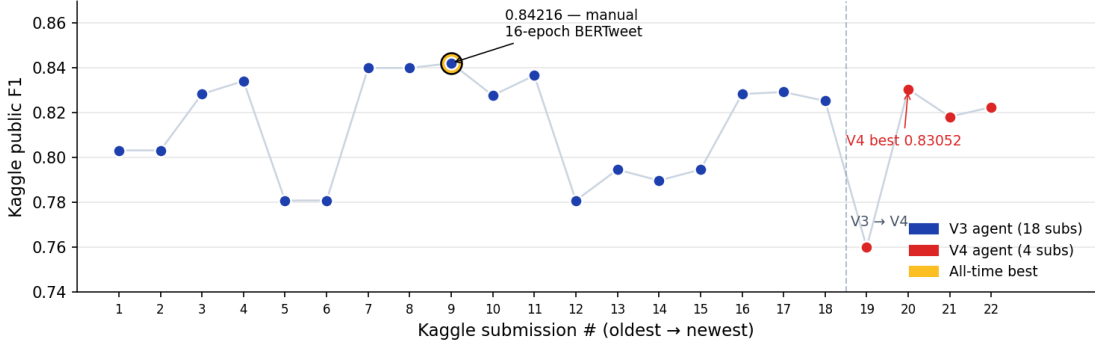


Figure 7: All 22 Kaggle submissions, oldest to newest, colour-coded by agent version. Dashed line marks the V3→V4 boundary (14 May 2026).

4 Reflections on Course Content

The seven families together cover every approach in the handout’s Track A table (Table 4); the remainder of this section discusses why each tier of families performs the way it does on this dataset.

Handout approach	Our family	Best V4 F1
Bag-of-words / n-grams + Dense MLPs	BoW, BoW_advanced	0.7296
1D Convolutions on text sequences	CNN	0.6753
LSTM / GRU / Bidirectional RNNs	LSTM	0.6207
Learned embeddings vs. pretrained (GloVe)	EmbeddingDL	0.7027
Transformer / self-attention	RoBERTa, BERTweet	0.7928

Table 4: Course-content coverage: every approach in the handout’s Track A table has a corresponding family in our agent. Best V4 F1 shown so the comparison is under the same compute budget.

BoW and BoW_advanced (the formula-based tier). Because BoW and BoW_advanced are formula-based representations, computed deterministically from term-frequency statistics via the TF-IDF weighting $\text{TF-IDF}(t, d) = \text{TF}(t, d) \cdot \log(N/n_t)$, we expect them to perform reasonably well on the disaster-tweets dataset. BoW applies the formula directly to word tokens; BoW_advanced extends the same idea with word and character n-grams plus separate channels for the keyword and location fields, stacked through several calibrated linear classifiers. Because the representation is parameter-free, these families have no embedding to underfit; they run fast with only a small trade-off in accuracy. We expect the advanced variant to outperform plain BoW by a small margin because character n-grams capture sub-word patterns that absorb the typos, hashtags and abbreviations typical of tweet text, the additional keyword and location channels add weakly correlated signal that single-channel TF-IDF cannot see, and stacking several linear models reduces the variance of any individual classifier, together adding roughly half a point to a full F1 point on top of the BoW baseline. The V4 numbers (0.7250 vs 0.7296) confirm this.

The transformer tier (RoBERTa, BERTweet, DistilBERT). The transformer families are based on pretrained contextual embeddings produced by a self-attention stack trained on billions of tokens before they ever see the disaster-tweets dataset, so we expect them to perform best on this task. Each loads a pretrained backbone from HuggingFace (roberta-base, vinai/bertweet-base, distilbert-base-uncased)

and fine-tunes the top layers on the 7,613 labelled tweets. The trade-off is computational: these backbones contain roughly 110 million parameters each, so while we expect them to score best, they are also the most expensive families in the sweep: at ≈ 500 s per trial on CPU they consume most of the per-family budget. BERTweet ultimately wins on Kaggle because of domain match: its tokenizer was pretrained on tweets and already handles @-handles, hashtags, contractions and the abbreviated style of Twitter text; RoBERTa and DistilBERT are architecturally sound but pretrained on Wikipedia and books, so they lose ground on tweet-shaped input.

The from-scratch deep tier (CNN and LSTM). CNN and LSTM rely on embeddings learned from scratch, initialised randomly and updated via backpropagation on only 7,613 labelled tweets, a corpus too small for those embeddings to converge to competitive vectors. What rescues the tier is architectural inductive bias: 1-D convolutions (course Lecture 2) act as learned local n-gram detectors, and gated recurrent cells (course Lecture 3) explicitly model sequential structure. Both extract usable signal even when the underlying embeddings are weak, which is why CNN (0.6753 V4 best) and LSTM (0.6207) sit above EmbeddingDL but well below the BoW baseline. The trade-off is unfavourable: higher compute cost than BoW for comparable or worse performance.

EmbeddingDL (the floor of the tier). EmbeddingDL uses the same from-scratch embeddings but provides no architectural fallback; it is essentially a small GRU over the learned vectors, with no n-gram detector and no contextual prior. With pure learning and nothing to lean on we expect it to perform worst among the deep families, and the V4 results confirm it (0.7027 best, lowest of the seven families).

5 Limitations and Future Work

Compute budget and validation noise. The 1-hour CPU budget forces a 3-epoch cap on transformer fine-tunes; the manual 16-epoch BERTweet run on GPU in §3 lifted Kaggle F1 by 0.002 with no other change, proving the cap is binding. The 400-row local slice also swings by ± 0.02 versus Kaggle on identical models; k-fold CV would fix it but does not fit the budget. **Memory and ensembling.** Cross-launch memory is surfaced only to the spec proposer, not the planner; a scoped, family-filtered planner memory would re-enable cross-launch learning. A geometric-mean ensemble across the top transformer runs would likely add 0.5 to 1 F1 points; we spent the time on the architecture sweep instead. **Removed families and remaining bugs.** DistilBERT was dropped for speed and DeBERTa because its classifier head never transferred through HuggingFace’s loading API; both could be reinstated with a higher sandbox timeout or a custom load path. The undefined-stratification-labels bug still appears occasionally despite the BoW autofix; inlining the line directly into the template would close it permanently. **Back-translation augmentation.** One further research direction would be applying back-translation through one or more pivot languages (Shleifer, 2019), a low-resource text classification technique shown to deliver up to 8% accuracy gains; this is most likely to lift F1 on our sparse and recurrent families where pretraining is weakest.

Reference. Shleifer, S. (2019). *Low Resource Text Classification with Backtranslation*. arXiv:1903.09244. <https://arxiv.org/abs/1903.09244>

6 Individual Contributions

The seven model families in the agent are BoW, BoW_advanced, CNN, LSTM, EmbeddingDL, RoBERTa, BERTweet (plus deprecated DistilBERT and DeBERTa retained in V2/V3 logs). **Niccolò Germani:** agent main loop, V4 sweep orchestrator, Sweep Planner and Spec Proposer prompts, constraint engine and cross-launch veto, short-term memory file, run-history instrumentation, report writing. **Simon Gühring:** RoBERTa and BERTweet hooks (shared HuggingFace template), family-specific autofix pass on transformer specs, Kaggle submission pipeline. **Rahul Rajesh:** BoW and BoW_advanced hooks, from-scratch deep families (LSTM, CNN, EmbeddingDL), per-family preflight rules. **Dominik Plöchl:** LLM/Ollama client, Code Generator, Repair LLM and Analyst prompts, sandbox subprocess and timeout handling, experiment-log analysis behind the figures in §2.

Resources. Video: <https://youtu.be/yUgWHV0mtu4> | Kaggle: [leaderboard \(search Tweets Master\)](#) | Code: <https://github.com/Nic000111/apa-disaster-tweets-agent>